RP8 — The source code of Revision_Deflation: what computes what, and how to compile and run it

Provenance required by Hard Rule 6 (PROJECT spacetimes, Revision_Deflation)

Prepared for Patrick Lee Nash, Ph.D.

November 12, 2026

1. What this document is for

Hard Rule 6 demands commented source code for every equation, result, figure and prediction, instructions for using it, a provenance for every computed value, and that the code be executed. This document is the map of the code of the revision: which program computes which file, which file feeds which figure and which table, how the programs are tested, and how to compile and run everything. It follows the provenance P8 of the main project, which the reader may consult for the general rules; they are the same here:

- **Rust for every computation.** All physics of the revision is computed by one Rust program, `deflation` (folder `code/deflation-rs`: 18 source files, about 6200 lines, 38 unit tests). It uses the program of the main project as a library and no other library.
- **Python for an independent verification, and for tables, figures and documents.** The central results are verified a second time by a Python script that shares no code with the Rust program and uses other methods (computer algebra instead of rational arithmetic; a general-purpose integrator and library Bessel functions instead of the integrator of the program).
- **No number is typed by hand.** Every table of a document is written by a script from a data file that the program wrote; every check quoted in a document is reproduced verbatim in an appendix.

Where the code was executed. On a Linux computer (x86-64) with rustc 1.97.0, Python 3.13 (numpy 2.5.3, scipy 1.18.1, sympy 1.14.0, mpmath 1.3.0, matplotlib 3.11.2), pandoc and XeLaTeX. As for the main project, the assistant could not run the Rust program on the author's Mac, where no Rust compiler is reachable from its sandbox; the program has no dependency outside the repository and should build and run on macOS unchanged.

2. How to compile and run everything on a Mac

Each block can be pasted into the Terminal as it is. A line that begins with # is an explanation and does nothing. The Rust compiler and the Python packages are installed as described in P8 of the main project, Sec. 2, Steps 1 and 2; if the main project has been run on this Mac, nothing needs to be installed. The independent check needs two Python packages more than the main project:

```
# once: the two additional Python packages of the revision (computer algebra; precise special
 ↪  functions)
python3 -m pip install --user sympy mpmath
```

Step 1: go to the folder of the revision and build the program.

```
# every command of the revision is run from this folder
cd "/Users/nsh/Developer/github/spacetimes/Revision_Deflation"
# compile the program (the first time this also compiles the program of the main project; about a
 ↪  minute)
```

```
cargo build --release --manifest-path code/deflation-rs/Cargo.toml
# run the 38 unit tests; the last lines must say "test result: ok. 38 passed"
cargo test --release --manifest-path code/deflation-rs/Cargo.toml
```

Step 2: run the nine commands. Each prints its checks, one per line, beginning with `[PASS]` or `[FAIL]`, and ends with a line such as `matrices: ... checks passed`. Together they take about three and a half minutes and rewrite every file of the folder `data`.

```
# geometry, Dirac matrices, spin connection, Dirac operators           (less than a second)
./code/deflation-rs/target/release/deflation matrices
# solutions of the Dirac equations: creation and extinction           (about 12 seconds)
./code/deflation-rs/target/release/deflation modes
# gap, bound pair, matter, vacuum, temperature along the clock        (about a minute and a half)
./code/deflation-rs/target/release/deflation history
# can the paired state follow the deflation?                          (about 16 seconds)
./code/deflation-rs/target/release/deflation quench
# Hubble constant, bounds on the rate, the exponent under H2          (about a minute)
./code/deflation-rs/target/release/deflation cosmology
# surviving symmetries, vortex zero modes, exponents of the couplings (less than a second)
./code/deflation-rs/target/release/deflation symmetry
# exchange amplitudes, modes of the scalar                            (about a second)
./code/deflation-rs/target/release/deflation channels
# dark matter, growth of structure, drifts                            (about 6 seconds)
./code/deflation-rs/target/release/deflation matter
# the paired state from the effective action                          (about 10 seconds)
./code/deflation-rs/target/release/deflation paired
```

Step 3: the independent verification (about 6 seconds; must end with “33 of 33 checks passed”).

```
python3 code/scripts/verify_deflating_independent.py
```

Step 4: symbols, tables, figures, documents.

```
# the registry of symbols and its six checks (provenance RP1)
python3 code/scripts/write_symbol_registry.py
python3 code/scripts/check_symbols.py
# the tables of the documents, from the data files
python3 code/scripts/write_doc_tables.py
# the fourteen figures (written to the folder figures as .pdf and .png)
python3 code/scripts/make_figures.py
# the documents: Markdown from the sources, then LaTeX and PDF (needs pandoc and XeLaTeX)
python3 code/scripts/assemble_docs.py
bash code/scripts/build_docs.sh
# the two manuscripts (needs pdflatex with REVTeX 4.2)
python3 code/scripts/build_manuscripts.py
```

If a command of Step 2 prints a line that begins with `[FAIL]`, or if the numbers in a document change when Steps 2 to 4 are repeated, something is wrong and should be reported; on another processor the last digits of floating-point results may differ, the exact checks may not.

3. The Rust program, file by file

All files are in `code/deflation-rs/src`. Every file begins with a description of the physics that it computes, with the formulas; every public function is documented.

file	lines	content
`rat.rs`	223	exact rational numbers with overflow checks; the trait that lets the geometry be computed in rational or in floating-point arithmetic
`qmat.rs`	201	exact matrices with complex-rational entries

file	lines	content
metric.rs	714	the deflating metric: Christoffel symbols, Riemann, Ricci and Einstein tensors from their definitions and in closed form; Killing vectors; Lie derivative
geometry.rs	443	the exact geometry checks G1 to G7, G10 (Lovelock tensors) and the geodesics G8, G9
curved.rs	530	coordinate matrices, spin connection, Dirac operators; checks C1 to C8, D1 to D7
modes.rs	766	integration of the mode equations (commutator-free Magnus method), creation of particles, bounded solutions, modified Bessel function of complex order
scalar.rs	295	modes of a scalar field: exponents, static source, creation, bounded solutions
history.rs	274	the couplings along the clock; thresholds and events
quench.rs	280	time-dependent mean-field dynamics of the shell model
cosmo_defl.rs	346	expansion with deflation, sound horizon, age, growth of structure
symmetry_defl.rs	217	test of the 28 rotations and boosts; algebra of the vortex zero modes
cmd_symmetry.rs, cmd_channels.rs, cmd_matter.rs, cmd_paired.rs	985	the commands symmetry, channels, matter, paired
main.rs	788	the commands matrices, modes, history, quench, cosmology
io.rs, lib.rs	106	writing of data files and reports; list of modules

What is taken from the program of the main project (crate spacetimes, used as a library and not changed): the Dirac matrices and their checks, the routine for the internal symmetry, the exchange amplitudes, the gap equation, the bound pair, the effective action, the routines for matter, temperature and vacuum, the background cosmology without deflation, and linear algebra. The revision calls these routines with the couplings of each moment; it does not copy them.

4. The checks

command	file of checks	number
matrices	data/flat_matrix_checks.txt (the checks of the main project, again)	106 of 106
matrices	data/deflating_checks.txt	47 of 47
modes	data/mode_checks.txt	7 of 7
history	data/history_checks.txt	8 of 8
quench	data/quench_checks.txt	5 of 5
cosmology	data/cosmology_checks.txt	6 of 6
symmetry	data/symmetry_checks.txt	18 of 18
channels	data/channel_checks.txt	7 of 7
matter	data/matter_checks.txt	6 of 6
paired	data/paired_checks.txt	7 of 7
Python script	data/deflating_checks_independent.txt	33 of 33
symbols	data/symbol_checks_deflation.txt	6 of 6

Each file of checks exists also as .csv. A check marked “exact” is an identity verified in exact rational arithmetic; a check marked “numerical” states its accuracy. The identifiers begin with

`44d-` (deflating 4+4), `71d-` (deflating 7+1) or `def-` (both); those of the Python script with `I`; those of the symbols with `RS`.

Was the independent script itself tested? Yes: twenty deliberate errors were introduced into it, one at a time (a wrong sign in a Christoffel symbol, a wrong factor in the Einstein tensor, the wrong order of a Bessel function, and so on). The script detected nineteen. The twentieth, a change of the term gprod in the equation of the expansion, cannot be detected by the comparison of Hubble constants, because that term is absorbed by the cosmological term (RS3, Sec. 2); it is tested by the symbolic check of the Einstein tensor instead. The checks of the completeness of the Killing vectors and of the Lanczos tensor were added afterwards; the second is validated by the agreement of two independent formulas (the program uses the formula for curvatures that act on pairs of directions, the script the standard formula with four terms).

5. The scripts

All in `code/scripts`.

script	what it does
`verify_deflating_independent.py`	the independent verification (Sec. 4)
`write_symbol_registry.py`, `check_symbols.py`	the registry of the symbols and its six checks (RP1)
`write_doc_tables.py`	writes every table of the documents from the data files (`provenance/generated/tables_deflation.md`)
`make_figures.py`	draws the fourteen figures from the data files
`assemble_docs.py`	inserts tables, registry and program output into the document sources (`docs_src/*.src.md`)
`build_docs.sh`, `doc_header.tex`, `check_layout.py`	Markdown to LaTeX to PDF, and a check that no line, table cell or formula is wider than its box
`build_manuscripts.py`, `tex2md.py`	the two manuscripts from one source; their Markdown versions

6. Which data file feeds which figure

figure	data files	in
`fig_geodesics`	`geodesic_1671d.csv`, `geodesic_1644d.csv`	DMD
`fig_isometries`	`isometries.csv`	DMD, RS2
`fig_creation`	`creation_single.csv`, `creation_history.csv`, `creation_anisotropic.csv`, `boson_creation.csv`	DMD
`fig_extinction`	`extinction_profile.csv`, `boson_bounded.csv`	DMD
`fig_clock`	`coupling_exponents.csv`, `events.csv`	RS1
`fig_scalar`	`scalar_zero_modes.csv`, `static_source.csv`	RS1
`fig_kernels`	`pair_channel_kernel.csv`	RS1
`fig_gap_history`	`gap_history.csv`	RS1
`fig_binding`	`dimer_history.csv`, `dm_resonance.csv`	RS1, RS3
`fig_quench`	`quench_history.csv`, `quench_summary.csv`	RS1
`fig_cjt_history`	`cjt_history.csv`	RS4
`fig_matter_history`	`medium_history.csv`, `matter_saturation.csv`, `vacuum_history.csv`, `thermal_history.csv`	RS4
`fig_hubble`	`hubble_deflation.csv`, `hubble_mass_loss.csv`, `growth_deflation.csv`	RS3

figure	data files	in
`fig_bounds`	`deflation_bounds.csv`	RS3, RS2

Every data file begins with lines that state which program and which command wrote it and what its columns mean.